METROINSIGHTS

Technologies, Fundamentals and Design Decisions

Phase 3 study material | Repository verified 14 October 2026

Purpose: prepare you to explain what each important technology does, how MetroInsights uses it, why it fits, where it falls short, and when another design would be better.

Evidence rule: each claim is anchored to repository implementation or is clearly identified as a general concept. Package versions listed here come from requirements.txt; they are declared constraints, not proof of which versions are running in a deployed environment.

Repository: `dmrc_footfall_predictor` | Branch `main` | HEAD `4405d16`

How the stack fits together

MetroInsights keeps the runtime simple: a Python web process loads a trained estimator and small files, exposes a JSON endpoint, then the browser renders the result. Data generation and model training are offline scripts.

Concern	Technology / design	Project evidence
HTTP application	Flask + Jinja2 + Gunicorn	app/__init__.py, app/routes.py, templates, Procfile
Tabular data / features	pandas + NumPy	CSV ingestion, feature row assembly, trigonometric time encodings
Demand model	scikit-learn HistGradientBoostingRegressor	pipeline/03_train_and_evaluate_forecaster.py; saved joblib artifact
Persistence / configuration	CSV + JSON + joblib files	data/cleaned, data/synthetic metadata, models/; no database
Network travel estimate	Weighted graph + Dijkstra + Haversine	network_topology.py and travel_graph.py
Browser UI	Vanilla JavaScript + Chart.js + HTML/CSS	dashboard.js, Jinja templates, Chart CDN reference
Production process	Gunicorn WSGI server	Procfile: web: gunicorn run:app

Declared dependency detail: requirements.txt pins Flask 3.1.3, Jinja2 3.1.6, NumPy 2.5.2 and pandas 3.0.5; scikit-learn is specified as >=1.5.0, joblib >=1.4.0 and SciPy >=1.10.0; Gunicorn is listed without a version. Repository search finds no direct SciPy import in app or pipeline code. These declarations are not a lockfile and do not prove the deployed runtime versions.

A useful design explanation

“The architecture separates offline model building from online inference. Flask handles requests and composes the forecast, schedule scenario and graph estimate. Small static artifacts are sufficient for serving; generated training rows are needed only to rebuild the estimator. The front end is intentionally light, but the API/data contract is maintained manually.”

One boundary to state early

The project’s model and operational parameters are built from synthetic or hand-authored data. The stack demonstrates a demand-planning workflow; it does not provide validated live passenger counts or dispatch decisions.

1. Python and modular application design

Core concept

Python is a general-purpose, dynamically typed language. Modules define namespaces; imports connect functionality. Functions let the code isolate operations such as parsing, feature creation and graph search.

How MetroInsights uses it - implementation evidence

The runtime is split across `app/routes.py`, `forecasting.py`, `scheduling_optimizer.py`, date/service helpers and the graph module. `run.py` imports the Flask app; paths are constructed from `__file__` so artifacts resolve relative to the repository.

Advantages and trade-offs

Why it fits: one language spans Flask, pandas data preparation and scikit-learn model training; broad libraries and readable iteration make a prototype easy to inspect. Trade-offs: dynamic typing and import-time file loads allow schema/path failures to surface at runtime or boot; large in-memory loops consume memory.

Alternatives and when they win

When another choice wins: a typed service with many teams may benefit from stricter typing/contracts; a high-throughput numerical kernel may use compiled/vectorized code; a multi-service platform may separate model serving from the web layer.

Interview questions and difficult follow-ups

Why one language for pipeline and app?
It reduces serialization and integration friction: both stages share Python objects and scikit-learn artifacts. It also couples runtime dependency versions to training artifacts.

What is the risk of loading files at import?
It simplifies one-time initialization and avoids per-request reloads, but missing/corrupt files can stop process startup and makes module imports side-effectful. A factory/lifespan loader can make initialization explicit and easier to test.

2. Flask, HTTP, JSON APIs and Jinja2

Core concept

Flask is a WSGI web framework. A route maps an HTTP method and URL to a Python function. A request carries input; the handler validates it and returns an HTTP status plus a representation such as JSON or HTML. Jinja renders server-side HTML templates.

How MetroInsights uses it - implementation evidence

Evidence: `app/__init__.py` creates Flask; `routes.py` registers `/dashboard`, `/api/predict`, station and metrics routes. The prediction endpoint accepts GET/POST; the browser POSTs JSON. Landing/dashboard templates are rendered by Flask/Jinja.

Advantages and trade-offs

Benefits: low ceremony, easy route-level orchestration, built-in request/response and template integration. Costs: validation and API schemas are mostly hand-coded; synchronous WSGI can block a worker during slow computation; no built-in database or auth policy.

Alternatives and when they win

FastAPI wins for type-derived OpenAPI schemas, request validation and async integrations. Django wins when admin, auth, ORM and batteries-included workflows matter. Flask stays sensible for a small synchronous dashboard.

Interview questions and difficult follow-ups

What makes an API RESTful?
HTTP resources, methods, status semantics and stateless requests are useful REST principles. This app has JSON HTTP endpoints, but do not overclaim strict REST resource modeling.

How should invalid station input be handled?
Validate against station metadata at the boundary and return a stable 4xx JSON error. Avoid letting a downstream graph lookup fail as an unexplained 500.

What changes if inference takes seconds?
Measure latency; add worker capacity or move computation to a background/task service if it is long-running. Do not assume async alone makes CPU-bound model inference faster.

3. WSGI and Gunicorn deployment

Core concept

WSGI is the Python interface between a synchronous web application and a server. Gunicorn starts worker processes and passes HTTP requests to the WSGI callable. The application object is exported as `app` from `run.py`.

How MetroInsights uses it - implementation evidence

Evidence: `Procfile` runs `gunicorn run:app`; `run.py` exposes Flask and reads PORT for local execution. The Gunicorn command loads module `run`, then callable `app`.

Advantages and trade-offs

Benefits: production-grade process management compared with Flask development server; worker processes can handle independent requests. Trade-offs: each worker may load its own model and graph, increasing RAM; free hosting may sleep; concurrency and worker count must match CPU/RAM. No explicit worker configuration or health-check policy is present.

Alternatives and when they win

ASGI servers (Uvicorn/Hypercorn) win for async protocols and high-concurrency async I/O. They do not automatically speed CPU-heavy synchronous model prediction. A managed model endpoint wins if the estimator lifecycle/scaling should be independent of the web app.

Interview questions and difficult follow-ups

Why not use app.run in production?
It is the development server; production WSGI servers manage worker processes and production request serving.

How many workers?
Benchmark on the chosen memory/CPU plan. If each process loads a large estimator, more workers can exhaust RAM. Tune using measured latency, memory and concurrency.

Will changing service region change the URL?
Usually service display name and generated hostname are separate settings; region migration can require creating/moving a service and can affect URL/latency. This is deployment configuration, not a code-level Flask feature.

4. pandas and CSV/JSON file storage

Core concept

pandas provides labeled Series/DataFrame structures and CSV readers/writers, useful for tabular cleaning and feature construction. JSON is a convenient format for small nested configuration/metadata. File-based storage is static persistence rather than a transactional database.

How MetroInsights uses it - implementation evidence

Evidence: `pd.read_csv` appears in cleaning, training, runtime station loading and operations loading. Forecast code creates a one-row DataFrame to preserve feature names/order. JSON stores line-distribution metadata, typologies and evaluation metrics. There is no database or ORM.

Advantages and trade-offs

Benefits: accessible data inspection, easy versioning of small inputs, simple deployment with a few static artifacts. Costs: full tables can be held in memory; CSV has weak schema/type guarantees and poor concurrent-write semantics; duplicate legacy outputs can drift; no query indexes, transactions or history.

Alternatives and when they win

SQLite wins for a small local transactional database. PostgreSQL wins for concurrent writes, relational constraints and querying. Parquet wins for large analytical columnar datasets. Object storage or an artifact registry wins for large generated training files.

Interview questions and difficult follow-ups

Why is the 130 MB CSV not needed at inference?
The loaded model/metadata and small runtime files provide what the prediction function uses. The CSV is an input to retraining, not queried during a normal request.

What must change to persist user scenarios?
Add a durable store, schema, migrations, transaction/error handling, retention policy and appropriate access control; local ephemeral filesystem storage is not reliable on many hosts.

Why not use pandas for every API operation?
A one-row DataFrame is convenient to keep feature names/order. For high throughput, measure allocation overhead and consider a pipeline or array path that preserves schema safely.

5. NumPy, randomness and time features

Core concept

NumPy provides numerical arrays, vectorized math and pseudorandom number generators. Cyclical features map periodic values to a circle: sin(2*pi*x/P), cos(2*pi*x/P), preventing a false numeric gap between the period's endpoints.

How MetroInsights uses it - implementation evidence

Evidence: generator uses `np.random.default_rng(seed)` and Poisson sampling; forecasting uses sine/cosine encodings for hour, weekday and month. The feature values become model inputs.

Advantages and trade-offs

Benefits: fast numerical operations, reproducible seeded generation and smooth periodic representation. Trade-offs: a fixed seed makes a run repeatable but does not validate its realism; date/event assumptions and distribution choice still govern synthetic labels. Sine/cosine features cannot by themselves express every calendar exception.

Alternatives and when they win

For irregular categorical periods, one-hot or learned embeddings can win; Fourier features can represent richer seasonal shapes; observed calendar/event covariates help when actually available. For generation, calibrated empirical distributions beat invented multipliers if trusted data exists.

Interview questions and difficult follow-ups

Why encode hour with sine and cosine?
So 23:00 and 00:00 are neighbors in feature space rather than far apart as raw integers. Both coordinates are needed to identify position on the circle.

Does seed=42 make the dataset scientifically reproducible?
It makes pseudorandom draws repeatable for fixed code/library inputs. It does not establish that the assumed rates or Poisson distribution model reality.

Why Poisson?
It generates nonnegative integer counts with mean and variance tied to the rate parameter. Real transit counts may be overdispersed, correlated or constrained, so fit/validate the count model against observations.

6. scikit-learn and gradient-boosted trees

Core concept

scikit-learn provides estimators with a common fit/predict interface and evaluation utilities. Gradient boosting builds an additive ensemble sequentially, where later trees correct residual errors. Histogram gradient boosting bins numeric features and can train efficiently on tabular data.

How MetroInsights uses it - implementation evidence

Evidence: pipeline trains `HistGradientBoostingRegressor` and a Ridge baseline; saved estimator is loaded in `forecasting.py` and invoked with `predict`. Hyperparameters are explicit: 250 iterations, learning rate .06, max depth 9, minimum leaf 25, L2 .05, seed 42.

Advantages and trade-offs

Why it fits: nonlinear feature interactions and tabular data; little scaling required for tree splits; missing values are supported by this estimator. Trade-offs: less interpretable than a linear model; a model can learn synthetic-generator artifacts; categorical IDs treated numerically can imply artificial ordering; saved artifacts depend on compatible libraries.

Alternatives and when they win

Ridge wins for a simple interpretable linear baseline and stable extrapolation. Random forests win when bagging robustness and less sequential fitting are attractive. CatBoost can be strong with high-cardinality categoricals if adding dependency is acceptable. A count distribution model may better represent count variance/uncertainty.

Interview questions and difficult follow-ups

Why compare a baseline?
It answers whether model complexity adds value over simple station/history rules. A sophisticated model should beat a relevant baseline on a valid holdout.

How do you encode station identity?
Here, integer indices plus historical aggregates. The integer id is a categorical token in intent, but a tree can split by numeric thresholds. One-hot/target encoding or categorical-aware estimators are alternatives; evaluate unseen stations separately.

Can you call this production validated?
It is the selected estimator used by the app, but model quality is validated only on generated holdout data. “Production model” in a source comment means serving artifact, not proof of real-world performance.

7. Statistics, validation and metrics

Core concept

A chronological holdout tests later dates after training on earlier dates. MAE is average absolute error in target units. RMSE is square-root mean squared error and penalizes larger misses more. R-squared compares residual error to variation around the target mean. MAPE averages relative absolute error, but is undefined at zero.

How MetroInsights uses it - implementation evidence

Evidence: pipeline splits before 2026-10-01 vs later dates; `compute_metrics()` reports MAE/RMSE/R2/MAPE and excludes zero targets for MAPE. Historical aggregates are designed from earlier rows/training-period history.

Advantages and trade-offs

Benefit: temporal split reflects forecasting direction better than random mixing. Trade-offs: one split can be season-specific; synthetic train/test are generated by the same code, so evaluation can reward recovering the simulation recipe. MAPE's nonzero-only subset can bias interpretation and underweight low counts.

Alternatives and when they win

Rolling-origin evaluation wins when measuring stability over multiple forecast dates. Pinball loss/quantile metrics win for uncertainty intervals; Poisson deviance can align better with counts; calibration and decision-cost evaluation matter if staffing decisions depend on thresholds.

Interview questions and difficult follow-ups

What does R2=0.9947 mean here?
The model explains most variation in the generated holdout labels under that split. It does not establish accuracy on observed ridership.

Why not random split?
Neighboring dates/hours can share patterns. Random split can leak near-duplicate temporal structure into both sets and overstate future performance.

How to validate a real deployment?
Acquire timestamped observed counts, define target/aggregation and leakage-safe dates, compare against operational baselines, segment errors by station/hour/date type, and monitor drift.

8. Joblib and model artifact persistence

Core concept

An estimator learned in training can be serialized to a file and loaded by the inference process. joblib is commonly used for Python objects containing NumPy arrays. Metadata is also stored to preserve feature columns, indices and historical means.

How MetroInsights uses it - implementation evidence

Evidence: training calls `joblib.dump` for model and metadata; `forecasting.py` calls `joblib.load` during module import. JSON metrics are loaded separately.

Advantages and trade-offs

Benefits: avoids retraining at app start and keeps inference fast; metadata supports training-serving feature alignment. Trade-offs: Python/library/version coupling, binary artifact reviewability, artifact size, and security risk because pickle-like formats can execute code when loaded. Artifacts must come from trusted source.

Alternatives and when they win

ONNX wins for cross-language inference portability when operator coverage supports the model. A model registry wins for versioning, approvals and rollback. Rebuilding from source wins when full provenance matters, but may be too costly for each web boot.

Interview questions and difficult follow-ups

What if model and metadata mismatch?
Feature missing/key errors or, worse, silent incorrect column interpretation. Version the pair together, validate schema/hash at startup, and deploy atomically.

Why is joblib not a data interchange format?
It is Python-object serialization, not a stable language-neutral schema like JSON/ONNX; loading assumes compatible Python classes and trusted bytes.

Why not load every request?
Repeated disk I/O and deserialization are wasteful. Process-level loading is efficient, but every Gunicorn worker may have its own copy.

9. Graphs, Haversine and Dijkstra

Core concept

A weighted graph has vertices connected by edges with costs. Dijkstra finds shortest paths from source(s) when edge costs are nonnegative, using a priority queue to expand the closest unsettled vertex. Haversine computes great-circle distance between latitude/longitude points on a sphere.

How MetroInsights uses it - implementation evidence

Evidence: nodes are `(station, line)`; adjacent stops connect bidirectionally; transfer edges carry fixed penalty. Haversine-derived minute weights use 32 km/h and outlier/missing-coordinate fallbacks. `shortest_times_from()` uses `heapq` multi-source Dijkstra.

Advantages and trade-offs

Benefits: topology constrains routes to connected line sequences; Dijkstra handles nonuniform transfer/running costs and provides a shortest-time estimate. Trade-offs: coordinates are not rail distances; constant speeds, fallback times and transfer penalty are coarse; no timetable wait or path reconstruction; graph may be stale/incomplete.

Alternatives and when they win

A* wins for repeated point-to-point queries on a large graph with a useful admissible heuristic. Time-dependent shortest path wins when schedules vary by time. GTFS-based routing wins when verified stop sequences and schedules are available. BFS wins only when all edges have equal cost.

Interview questions and difficult follow-ups

Why expand a station into station-line nodes?
It distinguishes being on different lines at the same station, so changing lines can incur transfer cost instead of being free.

What is Dijkstra complexity?
With adjacency lists and a binary heap, commonly O((V+E) log V); memory O(V+E). Exact details depend on heap implementation and stale entries.

Is the result a passenger travel itinerary?
No. The implementation returns best minutes per destination only. It omits waits and does not reconstruct station sequences.

10. JavaScript, fetch, Chart.js and accessibility

Core concept

JavaScript runs in the browser, can listen for input events, issue asynchronous HTTP requests with fetch, and update the DOM. Chart.js maps arrays of labels/data to canvas visualizations. Jinja sends an initial HTML shell from the server.

How MetroInsights uses it - implementation evidence

Evidence: `dashboard.js` initializes listeners, POSTs JSON to `/api/predict`, ignores stale requests by sequence number and renders KPIs/tables/charts. Chart instances are destroyed before recreated. Templates load Chart.js from a CDN.

Advantages and trade-offs

Benefits: no frontend build toolchain, fast iteration and server-rendered first page. Trade-offs: manual API contract, state spread between DOM and JS, CDN availability, and dynamic `innerHTML` requires escaping. Canvas charts need accessible text/table equivalents.

Alternatives and when they win

React/Vue win for complex reusable stateful interfaces and component testing; local vendored chart libraries improve CDN independence; SVG wins for bespoke accessible visualizations. Native HTML tables remain useful for exact values and screen readers.

Interview questions and difficult follow-ups

Why a request sequence id?
If response A takes longer than newer response B, ignoring A prevents stale data repainting the UI.

Does fetch cancellation remove server work?
AbortController can stop client waiting/request handling, but the server may already be computing; cancellation semantics depend on server and code.

What is the innerHTML risk?
Unescaped data can become script-capable markup. Use textContent for plain text or escape/allowlist values before creating HTML.

11. Scheduling rules and operations research basics

Core concept

A headway is the time between consecutive trains; trains per hour is 60/headway. Capacity utilization compares demand with a chosen capacity denominator. A constraint-based heuristic computes a feasible-looking scenario from business rules; it is not a global optimization solver unless it defines and solves an objective.

How MetroInsights uses it - implementation evidence

Evidence: `scheduling_optimizer.py` loads per-line CSV assumptions, bounds frequency from fleet/turnaround and signal/policy limits, uses peak/off-peak through-load factors, compares demand to boarding clearance, rounds practical headway, then derives wait and utilization.

Advantages and trade-offs

Benefits: explainable and fast, transparent scenario rules, easy to demo. Trade-offs: assumptions dominate answers; rounding and local decisions may be suboptimal; does not model train occupancy, OD flows, stochastic arrivals or corridor-wide conflict constraints.

Alternatives and when they win

Linear/integer programming wins for explicit fleet allocation and network-wide constraints; simulation wins for queueing/occupancy/dwell interactions; robust optimization wins under uncertain demand. These require better data and clearly specified objective/constraints.

Interview questions and difficult follow-ups

What is utilization denominator?
In this code it is estimated boarding-clearance capacity, adjusted for assumed through-load, not the train's physical capacity or observed onboard load.

How does headway relate to wait?
Under random passenger arrival and regular service, expected wait is approximately half the headway. This is an assumption; bunching and scheduled arrivals break it.

Why not call it an optimizer?
The module name uses optimizer, but behavior is rule-based frequency adjustment with constraints, not a mathematical solver over a network objective.

12. Date/time, timezone and operating-window design

Core concept

A timestamp is an instant or local wall-clock reading; these are different concepts. A time zone supplies offset and daylight-saving rules. A service window is a business rule that decides whether the app treats a selected local time as operating.

How MetroInsights uses it - implementation evidence

Evidence: route defaults date via Asia/Kolkata context and parses date/time; calendar_context computes weekday/occasion flags; operating_hours gates 05:00-23:30. Profile has integer hours 05-23, while selected-time input supports minutes.

Advantages and trade-offs

Benefits: explicit centralized gate avoids scattered after-hours checks. Trade-offs: route dates/times may be naive local datetimes; browser presets use browser locale/time; current hours are global assumptions, not line calendars. Hourly profile bucket at 23:00 is not a half-hourly boundary forecast.

Alternatives and when they win

Use timezone-aware timestamps for cross-region users; line/day-specific service calendars when verified schedules exist; interval-based targets when users need 30-minute or 15-minute fidelity.

Interview questions and difficult follow-ups

Why is 02:02 demand zero if graph shows rides?
The graph’s x-axis is operating hours; 02:02 is selected-time demand and should be closed/zero. The daytime curve is a daily profile.

What boundary tests matter?
04:59/05:00 opening and 23:30/23:31 closing. Test both selected-time demand and schedule status.

What if deployed in a different timezone?
Use explicit ZoneInfo localization for Delhi service interpretation and return/display timestamps consistently; don’t rely on host/browser defaults.

13. Design choices, alternatives and improvement priorities

Core concept

Technology choice is the fit between problem constraints, team capability, workload and evidence quality. The best interview answer names both a benefit and a condition under which another approach becomes preferable.

How MetroInsights uses it - implementation evidence

The current architecture is a compact monolith with local artifacts, synchronous request composition, hand-authored operational configuration, a rule-based service scenario, and a topology graph. This limits infrastructure and keeps the project explainable.

Advantages and trade-offs

Priorities follow validity and reliability: observed data + provenance first; robust holdout evaluation and uncertainty next; request validation and automated boundary tests; artifact/schema versioning; then deployment monitoring and persistence if product requirements need them.

Alternatives and when they win

Avoid proposing Kubernetes, streaming, a database or deep-learning model only because they are fashionable. Introduce complexity when there is a concrete requirement such as concurrent scenario writes, live feeds, measured latency limits, or network-wide optimization.

Interview questions and difficult follow-ups

What is the most important limitation?
Data validity: synthetic target and hand-authored parameters constrain the meaning of any accuracy/service claim.

What would you keep?
A lightweight Flask interface, saved estimator and transparent baseline/rule comparison are reasonable for a small prototype.

How choose next architecture step?
Start with user and scale requirements; quantify latency, data update frequency, concurrency and reliability; select the smallest design that meets them.

Interview-ready design decision summary

Decision	Defensible short explanation	Caveat to state
Flask + Gunicorn	A small synchronous JSON dashboard needs little framework overhead; Gunicorn is the WSGI serving process.	No async/live streaming requirement is implemented.
pandas + NumPy	CSV tables and feature transformations are clear and productive in the Python ML ecosystem.	Large training data still consumes memory; schemas need explicit validation.
HistGradientBoostingRegressor	Nonlinear tabular regression baseline selected and persisted for quick inference.	The target is synthetic; reported holdout scores are not measured-ridership accuracy.
joblib artifacts	Prevents retraining on every app start/request and stores model metadata together.	Trusted files and compatible dependency versions are required.
CSV/JSON	Simple static station/operations/config data without transactional user records.	Use a database or data lake as concurrency/query needs emerge.
Dijkstra graph	Weighted, topology-aware shortest-time estimates with interchange costs.	Weights are approximations; no schedule waits or route itinerary.
Heuristic scheduling	Fast, transparent translation from assumed demand to headway scenarios.	Not a timetable solver and not onboard crowding calculation.
Vanilla JS + Chart.js	Light UI can call JSON API and render charts without a build pipeline.	Manual contracts and XSS-safe rendering need care.

Three rules for strong answers

• Name the actual code file/function, not just the library.

• Explain the mechanism in plain language before naming parameters.

• State one benefit, one trade-off and the evidence/limitation that bounds the claim.

Example: “We use Dijkstra on a line-expanded graph because transfer penalties need to depend on line changes. It returns a shortest weighted estimate efficiently for nonnegative edge weights. Our edge costs are coordinate-derived approximations, so this is not a timetable-aware journey prediction.”

End of Phase 3. Phase 4 has not been generated.